

Repair Request - Solution Architecture

MVP Technical Architecture + Performance / Observability Revision

Document Control	Value
Document ID	RR-ARCH-001
Version	1.1 - Performance Revision Candidate
Status	Ready for Architecture / Security Review
Revision Date	12 September 2026
Supersedes	RR-ARCH-001 v1.0
Revision Scope	Close performance architecture, worker, observability, regression and traceability gaps without changing business semantics.
Approval	Solution Architect / Security Reviewer / Project Owner - formal approval pending

Revision rule / กติกาการแก้ไข: เอกสารฉบับนี้เป็นการปิด Audit ด้าน Performance/Technical Design เท่านั้น ไม่เปลี่ยน State, Role, Guard, SLA, Permission, Notification Recipient หรือ Business Semantics ที่ล็อกไว้เดิม หากคำตัดสินด้าน Performance ใดไม่มีอยู่ใน Business Baseline จะระบุเป็น **Gap / Proposed Technical Decision / Pending Approval** และไม่ถือเป็น Requirement เดิมโดยอัตโนมัติ.

Document control note: หน้านี้เป็น Document Control ที่มีผลสำหรับ revision นี้และ supersede ค่า Version/Status บนหน้าปกของ source baseline ที่ถูกรักษาไว้ในหน้าถัดไปเพื่อ traceability.

Repair Request Solution Architecture

สถาปัตยกรรมโซลูชันระบบ Repair Request / MVP Technical Architecture

Document Control	Value
Document ID	RR-ARCH-001
Version	1.0 - Technical Architecture Baseline Candidate
Status	Ready for Architecture / Security Review
Revision Date	12 September 2026
Approval	Solution Architect / Security Reviewer / Project Owner - formal approval pending
Sources	RR-REQ-001 v1.5; BR-RR-BASELINE v1.5; RR-STS-001 v1.5; UC v1.6; RR-DD-001 v1.3; RR-TC-001 v1.2; RR-ERD/DBD/API/UI v1.1
Audience	Solution Architect, SA, Backend/Frontend Developer, DBA, QA, Security Reviewer, DevOps
Purpose	Lock technical architecture, component boundaries, security boundary, deployment topology and Sprint 0 foundation without changing approved business semantics.

Architecture rule: เอกสารนี้เป็น Technical Design Decision layer. State, Role, Guard, SLA, Notification recipients และ business semantics ต้องยึด Business Baseline เดิม หาก architecture decision ใดทำให้ business semantics เปลี่ยน ต้องเปิด Change Request ก่อนนำไปใช้จริง.

1. Architecture Scope and Drivers

สถาปัตยกรรมนี้รองรับ Repair Request MVP แบบ Responsive Web โดยแยก frontend, business API, background worker, transactional database, private file storage และ observability ออกจากกัน เพื่อให้พัฒนา ทดสอบ และ deploy ได้อย่างอิสระโดยไม่ย้าย business authority ไปไว้ที่ UI.

Driver	Architectural Response
Business state integrity	State-changing operations use explicit commands; backend validates state/guard/role and executes atomic transactions.
Security / multi-scope	Identity is authenticated externally; API derives tenant/user authority and enforces tenant/site/ownership/assignment server-side.
Concurrency	SQL Server rowversion mapped to ETag/If-Match; stale action returns 409 with no partial transition.
Traceability	Correlation ID links request, transaction, audit and outbox/notification.
Performance	Server-side filter/sort/paging, focused projections, indexes aligned to list/search, SLA, technician and notification paths.
Auditability	Append-only audit; no business delete; UTC timestamps; immutable terminal history.
File safety	Private object storage; metadata in DB; malware scan; only CLEAN files satisfy evidence.

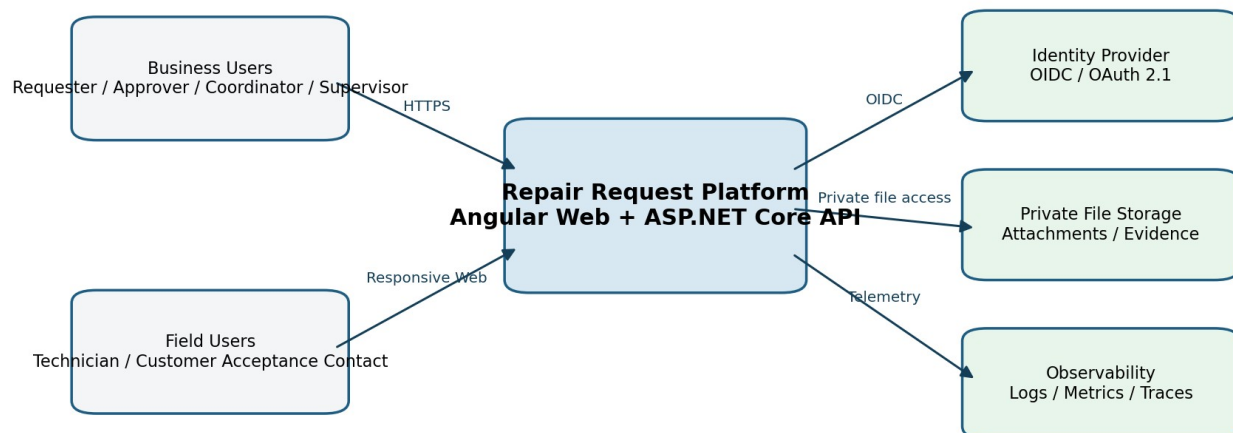
Portfolio / maintainability	Angular feature architecture + layered ASP.NET Core + EF Core + automated test + cloud deploy.
-----------------------------	--

2. Source Baseline and Non-negotiable Semantics

Baseline Area	Locked Semantics carried into Architecture
Identity	Requester = authenticated user / created_by; Customer/Company scope = server-derived tenant_id.
State	No direct state edit; no RETURNED state; no Reopen in MVP.
Acceptance	Exactly one designated Requester or active Site Contact may Accept/Reject.
SLA	24h 24/7 from SUBMITTED; Risk at T-4h; breach sticky; Time Correction never recalculates SLA.
MISSED	Original Visit stays MISSED; new linked Visit is optional; no-follow-up creates none.
Notifications	Recipient routing follows BR-18/D-14 matrix; delivery failure never rolls back committed business state.
Files	PDF/JPG/JPEG/PNG <=10 MB; private storage; scan required; CLEAN evidence only.

3. System Context

System Context - Repair Request MVP



The Repair Request platform is a browser-based internal/business application. Users interact through Angular; all authoritative business actions are validated by the ASP.NET Core API. External integration is out of MVP. Identity, file storage and observability are supporting platform capabilities, not alternate business authorities.

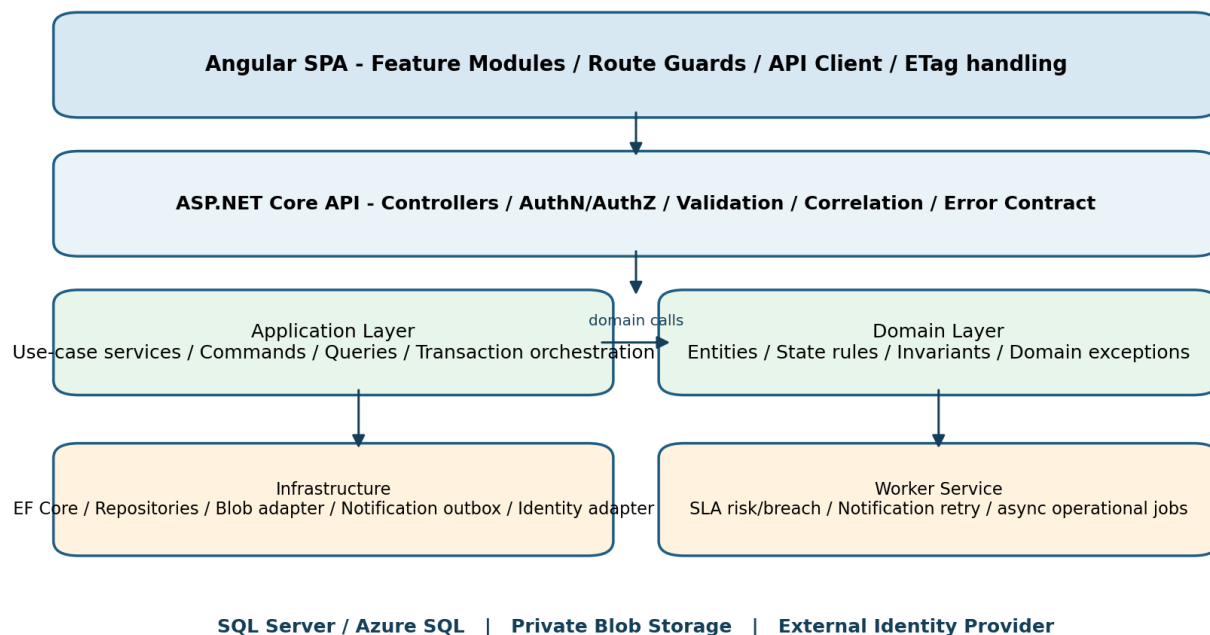
4. Technology and Platform Decisions

Area	Decision	Reason / Boundary
------	----------	-------------------

Frontend	Angular SPA, standalone/feature-oriented architecture	Matches current UI baseline; responsive desktop/mobile; reusable feature boundaries.
Backend	ASP.NET Core Web API on supported .NET LTS	Strong C# ecosystem, OpenAPI, middleware, async I/O, authorization and EF Core integration.
ORM	EF Core + SQL Server provider	Maps rowversion/transactions/constraints and supports focused LINQ projections.
Database	SQL Server locally; Azure SQL Database for cloud portfolio deployment	Keeps physical design consistent across development and hosted environment.
Background jobs	Separate .NET Worker Service	SLA monitoring and notification retry stay outside interactive request path.
File storage	Private object/blob storage behind adapter	Database stores metadata/reference only; no public object URL.
Auth standard	OpenID Connect / OAuth 2.x Authorization Code + PKCE for web client; API bearer access token	Avoid custom password logic in business API; provider remains replaceable deployment choice.
Observability	Structured logs + OpenTelemetry; cloud sink may be Application Insights	Trace API/DB/worker with correlation while avoiding sensitive payload logging.
CI/CD	Git-based pipeline; recommended GitHub Actions; IaC recommended for Azure resources	Repeatable build/test/deploy and portfolio reproducibility.
Version policy: exact Angular/.NET patch versions are locked at Sprint 0 to currently supported stable/LTS releases and recorded in Directory.Build.props/package-lock. Upgrading a framework version is a technical change unless it changes business behavior or API contract.		

5. Logical Application Architecture

Logical Application Architecture



5.1 Component Responsibilities

Component	Responsibility	Must NOT do
Angular SPA	UI state, forms, route-level UX, ETag caching, API calls, responsive rendering	Decide authorization, trust hidden buttons as security, write status directly.
API Layer	HTTP contract, authentication principal, authorization entry, validation envelope, correlation, ETag mapping	Contain complex business workflow or direct SQL.
Application Layer	Use-case orchestration, command/query handlers, transaction boundary coordination, notification/audit request creation	Depend on UI or cloud provider details.
Domain Layer	Entity invariants, state rules, business exceptions, domain value semantics	Reference EF Core, HTTP, Angular, Azure SDK.
Infrastructure	EF Core, repository/query implementations, storage, identity, notification, telemetry adapters	Invent business rules not present in source baseline.
Worker Service	SLA risk/breach detection, notification retry, asynchronous operational tasks	Perform user-authority actions without service-layer guard or bypass idempotency.

5.2 Backend Project Structure

Recommended solution structure:

```
RepairRequest.sln
src/
  RepairRequest.Api
  RepairRequest.Application
  RepairRequest.Domain
  RepairRequest.Infrastructure
  RepairRequest.Worker
tests/
  RepairRequest.Domain.Tests
  RepairRequest.Application.Tests
  RepairRequest.IntegrationTests
  RepairRequest.ApiTests
```

- Api references Application; Infrastructure is composed at startup through dependency injection.
- Application references Domain abstractions and ports; Infrastructure implements ports.
- Domain remains framework-light and owns canonical state/invariant logic where rules are aggregate-local.
- EF Core DbContext is the Unit of Work. Use focused repositories/query services where aggregate boundaries or reusable queries need abstraction; avoid a generic repository that only wraps DbSet without adding value.

6. Frontend Architecture

Area	Design
Feature modules	repair-requests, approvals, work-orders, service-visits, work-sessions, acceptance, corrective-actions, time-corrections, sla-reports, notifications.
Core	Auth service, route guards, HTTP interceptors, correlation/ETag handling, global error mapper, configuration.
Shared	Reusable controls, status badge, timeline, file uploader, validation summary, confirmation dialog, paging/filter components.
State management	Prefer feature-local state and RxJS/signals; introduce global store only when cross-feature complexity justifies it.
Security	Role/state visibility is UX only. Every API action is re-authorized by backend.

Concurrency UX	Store ETag with mutable resource; 409 forces reload/compare before retry.
Sensitive data	Do not persist Restricted PII, file URLs or access tokens in localStorage. Keep access-token handling in memory/session per selected OIDC library.

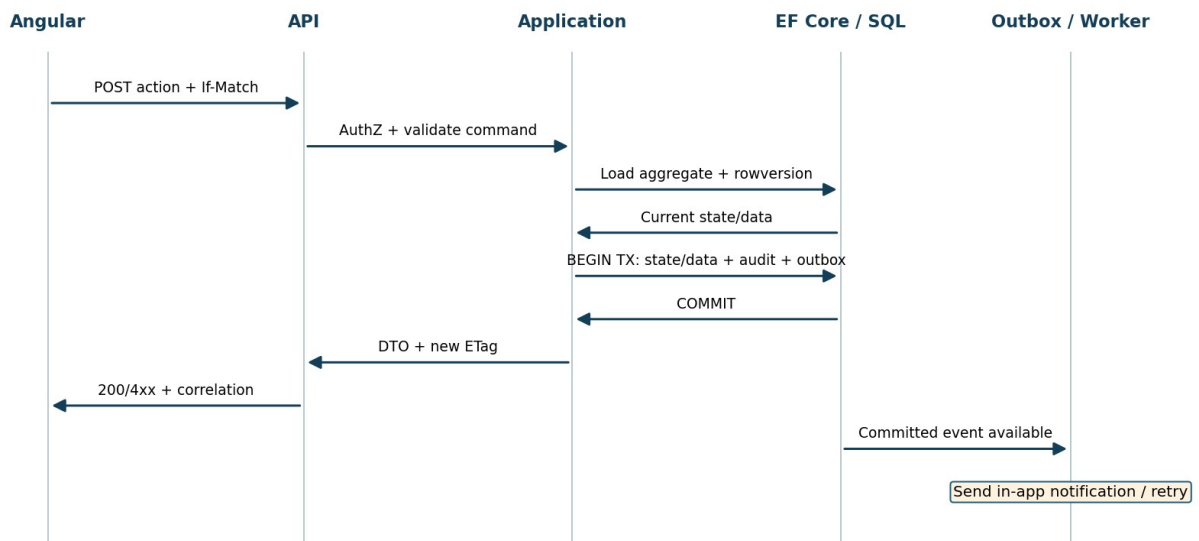
7. API and Command Architecture

REST endpoints remain aligned with RR-API-001 v1.1. Query endpoints are read-oriented; state changes are explicit command endpoints. Generic PATCH status or direct state mutation is prohibited.

Concern	Architecture Rule
Authentication	Authenticated principal required before business controller execution.
Authorization	Policy/service checks role, tenant/site, ownership/assignment and action permission.
Validation	DTO shape/format first; application/domain validation next; failure returns controlled 4xx.
Concurrency	If-Match required for mutable state commands; response returns fresh ETag.
Idempotency	Unique constraints + current-state/rowversion guard; optional idempotency key may be added for network-retry-sensitive creates without changing semantics.
Error contract	400 malformed, 401 no principal, 403/404 non-leaking denial, 409 state/concurrency, 422 business validation, 500 correlation only.
OpenAPI	Generated from API project and treated as implementation contract; CI detects breaking drift.

8. State-changing Transaction Pattern

State-changing Command Processing Sequence



Every significant action follows the same execution template: authenticate -> authorize -> validate current state/rowversion -> load required data -> execute one database transaction -> append audit -> insert outbox event/recipients -> commit -> return new ETag. Notification delivery occurs only after commit.

9. Data Architecture

Topic	Decision
Aggregate focus	Repair Request and Work Order are primary workflow aggregates. Service Visit/Session/Corrective/Acceptance remain explicitly linked and guarded by use-case transactions.
Keys	uniqueidentifier for transactional/control data; business numbers unique and immutable.
Concurrency	SQL Server rowversion on mutable aggregates; EF Core configured as concurrency token.
Time	datetime2(3) UTC in database; Site timezone converts only for presentation.
Queries	Use projection DTOs/AsNoTracking for list/search/read models; do not load full graphs for simple screens.
Indexes	Align with tenant/site/status/date, duplicate rule, SLA monitor, technician schedule, approval inbox, audit timeline and notification inbox/retry.
Deletes	No cascade path may remove Audit, Acceptance, Corrective, SLA breach or COMPLETED/MISSED history.
Migrations	EF Core migrations are versioned in source; production migrations run in controlled deployment step, not ad-hoc app startup.

10. Security Architecture Boundary

Control	Architecture Decision
Authentication	OIDC/OAuth authorization code + PKCE. Business API does not implement password verification.
Authorization	Server-side policies + use-case guard service. Claims identify principal; authoritative tenant/site/role/assignment data is resolved/validated server-side.
Tenant isolation	Every transactional read/action is scoped by tenant_id derived from authenticated context.
Site scope / IDOR	Resource lookup is combined with scope; guessed IDs outside scope return non-leaking denial/not-found.
Transport	HTTPS only outside local developer loop; HSTS in deployed environment.
CORS	Explicit trusted-origin allowlist. No wildcard origin with credentials.
Rate limits	Enable endpoint-class rate limiting for login-sensitive/API abuse and file upload; exact limits are deployment configuration.
Input	Allowlisted DTO fields; no mass assignment to entity/state/tenant/actor fields.
Secrets	No credentials/tokens in business tables or source control; use environment secret store / Key Vault.
Logs	Never log access tokens, passwords, Restricted PII, file content or sensitive error payloads.

Security review still required before Production: chosen Identity Provider, token lifetime/session strategy, CORS origins, rate-limit values, secret rotation, CSP/security headers and operational incident response must be approved in the Security Architecture / Deployment Plan.

11. File and Evidence Architecture

- Upload goes through authorized API. API validates extension/MIME/size and creates FileAsset metadata with PENDING scan status.
- Binary is stored in private object storage. storage_reference is internal; public object URLs are prohibited.
- Malware scan is asynchronous. Only CLEAN files can satisfy mandatory evidence or be served as usable evidence.
- Download is an authorized API action that rechecks tenant/site/role before returning or issuing a short-lived internal access mechanism.
- SHA-256 hash is retained for integrity/deduplication support. Retention duration is deployment policy, while Audit history remains protected from business deletion.

12. Notification / Outbox Architecture

Business transactions write NotificationEvent + deterministic NotificationRecipient rows in the same SQL transaction as the state change. A worker delivers in-app notifications after commit and retries FAILED/PENDING recipients. This preserves BR-18 while keeping provider failure outside the business transaction.

Stage	Behavior
Commit	State/data + audit + outbox event + recipient rows commit atomically.
Delivery	Worker claims pending recipient, writes SENT/FAILED, increments attempt count and timestamp.
Retry	Configurable backoff; repeated failure does not reverse business state.
Inbox	NTF API returns only current user's notifications under scope.
Future extension	Email/Line/message broker can be adapter additions; not part of MVP baseline.

13. SLA and Background Processing

Job	Algorithm / Idempotency
SLA Risk	Query active sla_record where risk_at <= now and risk_notified_at is null; emit NTF-SLA-RISK once and set risk_notified_at atomically.
SLA Breach	Query active sla_record where effective_due_at < now and breached_at is null; set breached_at once and emit breach event.
SLA Override	Synchronous authorized command updates effective_due_at + risk_at; existing breached_at is preserved.
Notification Retry	Retry PENDING/FAILED recipients under configurable attempt/backoff policy; delivery is independent of business state.

Polling interval is a technical configuration. A one-minute default is suitable for MVP unless operations requires a different cadence; changing cadence must not change the 24h/T-4h business semantics.

14. Logging, Audit and Observability

Signal	Use
Audit History	Business/legal trace: who did what, state before/after, reason, old/new and correlation. Append-only.
Application Log	Operational diagnostics; structured, no sensitive payload; includes correlation ID and safe entity identifiers.
Distributed Trace	HTTP -> application -> EF/SQL -> worker spans using OpenTelemetry.
Metrics	Request latency/error rate, DB latency, worker lag, notification failed count, SLA risk/breach count, file scan status backlog.
Alerts	Production thresholds defined in deployment runbook; alerting does not become a business state transition.

15. Performance and Scalability

The architecture must support the existing NFR baseline: normal List/Search ≤ 3 seconds under 100 concurrent users and 100,000 records per Site. The target is validated by TC-PERF-001; it is not guaranteed solely by the architecture document.

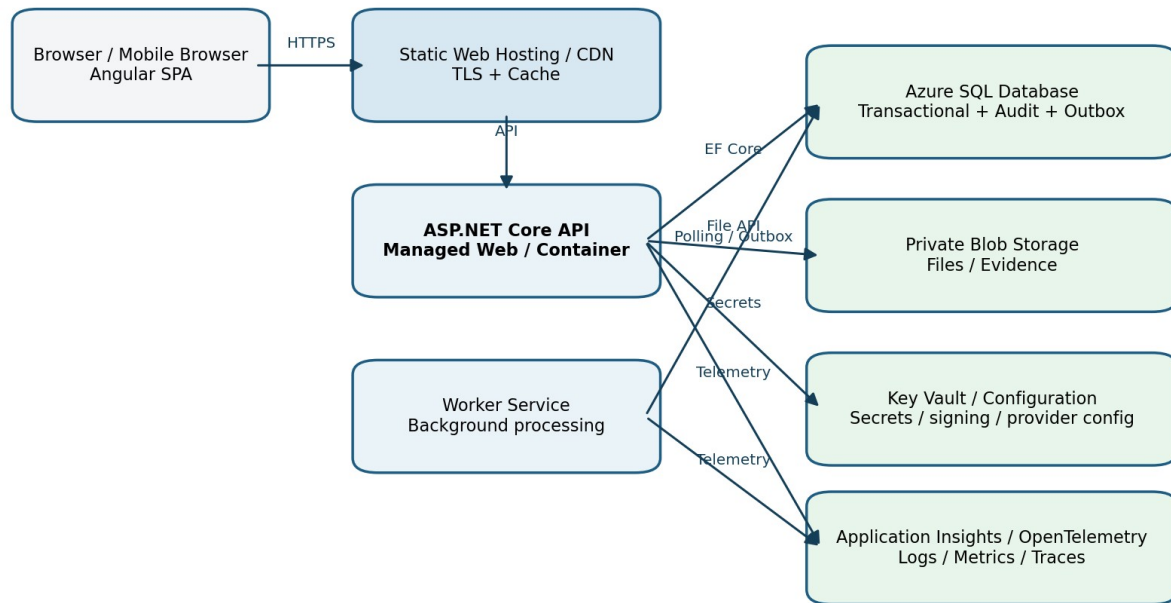
- All large lists use server-side filter/sort/paging; enforce maximum page size as deployment/product configuration.
- Use projected SQL queries and AsNoTracking for read paths; avoid N+1 and uncontrolled Include chains.
- Indexes follow RR-DBD-001 paths and are verified with realistic query plans/test data, not added blindly.
- Async I/O end-to-end for DB/file/network operations; no .Result/.Wait in request path.
- Do not introduce Redis/cache until measurement identifies a repeatable bottleneck that cannot be solved by query/index design. Cache must never bypass authorization scope.

16. Reliability and Failure Handling

Failure	Expected Architecture Behavior
DB transaction failure	Rollback entire state-changing command; return controlled 5xx/4xx with correlation; no partial state.
Concurrency conflict	409; no retry on stale business command until latest state reloaded.
Notification failure	Business success remains success; recipient remains retryable.
File scan failure	File remains unusable/rejected; does not corrupt business entity.
Identity provider unavailable	No new authenticated action; existing API request fails safely rather than accepting client authority.
Worker restart	Jobs are idempotent from durable DB state; pending rows remain available.

17. Deployment Architecture

Recommended Azure Deployment Topology



Environment	Recommended Topology
Local	Angular dev server + ASP.NET Core API + Worker + SQL Server/compatible local container + local private storage emulator/adaptor.
Dev/Test	Separate cloud resources or resource groups; Azure SQL; private Blob; test identity configuration; Application Insights.
Production-like portfolio	Static hosting/CDN, API managed web/container, separate Worker, Azure SQL, Blob Storage, Key Vault, telemetry. No public DB/storage endpoints unless explicitly secured.

18. Configuration and Secrets

- Use appsettings base + environment overrides for non-secret configuration. Secrets come from user-secrets/local environment in development and Key Vault/managed secret provider in cloud.
- Configuration includes allowed origins, identity metadata, DB connection, storage account reference, worker cadence, notification retry policy, page-size limit and telemetry endpoint.
- Business constants already locked by baseline (24h SLA, T-4h risk, state codes, role semantics) must not be silently overridden by deployment config unless the baseline explicitly marks them configurable.

19. Testing Architecture

Test Layer	Primary Coverage
Domain Unit	State rules/invariants/value logic without database/network.
Application Unit	Use-case validation/authorization orchestration with controlled fakes where useful.

Integration	EF Core mappings, constraints, rowversion, transactions, outbox, SQL behavior against real SQL Server-compatible test database.
API Contract	Endpoint/status/error/ETag/auth guard/OpenAPI compatibility.
Frontend	Component/form/action visibility/error/concurrency UX; visibility is not treated as security proof.
E2E / System	Cross-role workflow from Request through Close/Corrective/Time/SLA/Notifications.
Security	IDOR/cross-tenant/site, file access, mass assignment, stale/replay, sensitive error/log checks.
Performance	TC-PERF-001: 100 concurrent / 100k per Site / representative filter-sort-paging <=3s baseline.

20. CI/CD and Quality Gates

Stage	Required Gate
Pull Request	Restore/build Angular + .NET; lint/format; unit tests; dependency/security checks; migration compile check.
Integration	Start SQL test database; run EF/integration/API tests; generate OpenAPI and detect unintended contract drift.
Artifact	Version immutable frontend/API/worker artifacts; publish migration bundle/script separately.
Deploy Dev/Test	Apply infrastructure/config, run controlled migration, deploy API/worker/frontend, smoke test.
Production Gate	Formal baseline approval, Security review, UAT approval, backup/RTO/RPO/observability/provider settings approved.

21. Architecture Decision Register

ID	Decision	Status
ARCH-01	Angular SPA is the MVP web client; responsive design covers technician/acceptance mobile use.	Proposed
ARCH-02	ASP.NET Core Web API is the only business command authority.	Proposed
ARCH-03	Application/Domain/Infrastructure layered boundaries; no business rule in UI/controller.	Proposed
ARCH-04	EF Core + SQL Server/Azure SQL; SQL rowversion -> ETag/If-Match.	Proposed
ARCH-05	OIDC/OAuth authorization code + PKCE; external identity provider; no custom password auth in business API.	Security Review
ARCH-06	Private blob/object storage + asynchronous malware scan; DB stores metadata only.	Proposed
ARCH-07	DB-backed transactional outbox + NotificationRecipient; separate worker for delivery/retry.	Proposed
ARCH-08	Separate worker for SLA risk/breach and asynchronous operational jobs.	Proposed
ARCH-09	OpenTelemetry structured telemetry + correlation ID; cloud sink is deployment choice.	Proposed

ARCH-10	No distributed cache in MVP unless profiling demonstrates need.	Proposed
ARCH-11	Production database migrations run as controlled deployment step, not automatic application startup.	Proposed
ARCH-12	Azure deployment target: static frontend + API + Worker + Azure SQL + Blob + Key Vault + telemetry.	Proposed

22. Deferred Decisions / Production Gate

Deferred Item	Required Before
Exact Identity Provider and tenant onboarding method	Security implementation / Production
Token/session lifetime, CSP, security-header values	Security review
Exact CORS origins and rate-limit thresholds	Deployment
Blob malware scanning provider	Production evidence upload
Backup policy, RTO/RPO, retention/archive duration	Production
Alert thresholds and on-call/incident procedure	Production operations
Exact export format and max page size	Product/deployment configuration

23. Sprint 0 - Development Foundation

Workstream	Sprint 0 Deliverable
Repository	RepairRequest.sln + Angular workspace + branch/PR conventions + README/environment setup.
Backend	Api/Application/Domain/Infrastructure/Worker projects; DI; global error contract; correlation middleware; OpenAPI.
Frontend	Core/shared/layout skeleton; routing; API base client; auth placeholder/provider integration; global error handling.
Data	DbContext, baseline entity mapping, rowversion configuration, initial migration, seed/reference test data strategy.
Security	OIDC integration skeleton; policy/permission service; server-derived tenant/user context; non-leaking access behavior.
Observability	Structured logging + trace correlation + health endpoints.
Test	Unit/integration/API test projects; SQL test environment; first smoke test.
CI	Automated build/test for .NET + Angular; migration/OpenAPI checks.

23.1 Sprint 0 Definition of Done

- Angular shell can call an authenticated health/read API through environment configuration.
- API connects to SQL and EF Core migration can create baseline schema in a clean development database.
- Unauthorized and out-of-scope sample requests fail with controlled non-leaking response.
- Correlation ID appears consistently in API response/log and is available for audit/outbox linkage.
- ETag/If-Match concurrency proof-of-concept passes an integration test.
- Worker starts independently and can claim a test outbox/SLA row idempotently.
- CI pipeline builds Angular/.NET and runs unit/integration smoke tests successfully.
- No business feature is considered complete until the corresponding approved FR/BR/ST/UC/Test trace is implemented.

24. Traceability to Existing Technical Baselines

Architecture Area	Primary Source
State/Guard/Actor	RR-STS-001 v1.5; BR-RR-BASELINE v1.5
Use-case orchestration	UC-RR-001 v1.6
Entity/data/security classification	RR-DD-001 v1.3
SQL constraints/index/transaction boundaries	RR-DBD-001 v1.1
REST endpoint/error/concurrency contract	RR-API-001 v1.1
Screen/responsive/API mapping	RR-UI-001 v1.1
System/NFR/security/UAT acceptance	RR-TC-001 v1.2

25. Architecture Readiness Decision

Status: READY FOR ARCHITECTURE / SECURITY REVIEW.

The architecture is implementable for Sprint 0 and preserves the revised Repair Request business baseline. Development of business features should start after formal Business Baseline approval and Architecture/Security approval. Production deployment remains gated by identity-provider selection, security configuration, backup/RTO/RPO, malware-scan provider and observability/operations approvals.

Appendix A - Recommended Repository Layout

```

repair-request/
  frontend/          # Angular
  backend/
    src/
      RepairRequest.Api/
      RepairRequest.Application/
      RepairRequest.Domain/
      RepairRequest.Infrastructure/
      RepairRequest.Worker/
    tests/
      RepairRequest.Domain.Tests/
      RepairRequest.Application.Tests/
      RepairRequest.IntegrationTests/
      RepairRequest.ApiTests/
  infrastructure/    # IaC / deployment scripts
  docs/              # approved baseline + ADRs
  .github/workflows/ # CI/CD
  README.md

```

Appendix B - Implementation Guard Checklist

- No controller/action accepts tenant_id, actor_id, role or status as client authority.
- Every state-changing command has an explicit endpoint/use case and current-state guard.
- Every mutable aggregate returns ETag and validates If-Match where required.
- Audit + outbox rows are inserted inside the same transaction as the business state change.
- Notification send occurs after commit and cannot reverse a completed transaction.
- Files are private, scanned and authorized again at download.
- List/search/report queries apply tenant/site scope before paging and never filter sensitive data only on the client.
- No terminal state is reopened and no RETURNED state is introduced.
- SLA Risk remains T-4h and Time Correction never recalculates SLA.
- Automated tests map to the canonical Test Case/State/Use Case IDs rather than ad-hoc names.

Performance Audit Revision Appendix

ส่วนนี้เป็น revision content ที่มีผลเหนือ technical note เดิมเฉพาะเมื่อมีข้อความขัดกัน โดยไม่เปลี่ยน Business Baseline.

26. Performance Measurement Decision Boundary

PERF-DEC-01 - Performance Measurement Statistic (Pending Approval).
Business Baseline ระบุ “Normal List/Search <= 3 seconds at 100 concurrent users and 100,000 records per Site” แต่ไม่ได้ระบุว่า 3 seconds หมายถึง Average, Maximum, p95 หรือ p99. จึง **ห้ามถือ p95 เป็น Requirement ที่อนุมัติแล้ว**. Recommendation สำหรับการอนุมัติคือ p95 end-to-end API response time <= 3.0 seconds; ก่อนอนุมัติ ให้บันทึก Average/p50/p95/p99/throughput/error rate ครบและรายงานผลเป็น diagnostic evidence โดยยังไม่ประกาศ Pass/Fail ของ release gate จาก percentile ใด percentile หนึ่ง.

27. Worker Capacity / Claim Design

Worker remains separate from the interactive API. SLA/outbox processing uses durable database state and an atomic/idempotent claim so multiple worker instances cannot complete the same logical event twice. Poll cadence, batch size, degree of parallelism and retry backoff are **technical configuration** and must be tuned from worker-lag/DB-contention evidence. No fixed batch/concurrency value is promoted to a Business Requirement.

28. Observability for Performance

Signal	Required Evidence
HTTP/API	request count, error rate, Average/p50/p95/p99 latency, route label without sensitive values
Database	DB latency, slow/long-query evidence, CPU/resource utilization, connection-pool pressure where measurable, blocking/waits during concurrency run
Worker	lag, claimed/completed/failed, outbox/notification backlog, retry count
Files	upload/download duration/bytes, memory behavior and scan backlog
Trace	API -> Application -> EF/SQL -> Worker spans where applicable; business audit/outbox correlation preserved

Production alert thresholds/on-call procedure remain a Production Gate; missing exact alert values does not block Development instrumentation.

29. Performance Regression Protection

CI continues build/unit/integration/security/OpenAPI gates. Full load/stress/soak is not required on every PR. A versioned RR-PERF-001 run is required pre-release/before UAT sign-off for releases changing critical query, EF mapping, index, worker or Angular list path. Results are stored and compared with the last approved baseline.

30. Cross-Document Performance Traceability

Area	Business / NFR Source	Architecture / Design	Test / Evidence
List/Search	Requirement NFR <=3s @ baseline; statistic unresolved	Server-side paging, projection/ AsNoTracking, candidate indexed paths	TC-PERF-001/002 + RR-PERF-001; percentile gate follows PERF-DEC-01
Report/Export	FR-09 functional scope	API/UI report contract; execution strategy evidence-based	TC-PERF-003 rows/bytes/ time/memory + scope correctness
Worker	BR-17/18	Separate idempotent Worker + DB access paths	TC-PERF-004 lag/backlog/ retry/duplicate evidence
Files	<=10MB/private/CLEAN	Private object storage + authorized API	TC-PERF-005 duration/ memory/storage/security evidence
Concurrency	Reliability NFR / rowversion	ETag/If-Match + atomic transactions	TC-SEC-002 / TC-PERF-006 conflicts/waits/no duplicate

31. Scalability Boundary

No Redis, message broker, read replica, sharding or global cache is required merely to claim “high performance.” Introduce additional infrastructure only after repeatable measurement identifies a bottleneck and the design proves tenant/site isolation, invalidation/failure behavior and unchanged business semantics.